

Proyecto 3

Seguridad del Software — IC-8071

Instituto Tecnológico de Costa Rica

Documentación de verificación con OpenJML

Estudiante: Pablo Mesén Vargas
Carné: 2023071259
Estudiante: Alonso Durán
Carné: 2023044780

Índice

1. ¿Descubrió OpenJML todos los posibles problemas? ¿Es el código correcto después de los cambios?	2
1.1. Archivo Bag.java	2
1.2. Archivo Amount.java	3
2. Sugerencias de mejora para OpenJML y JML	3
2.1. Para OpenJML (la herramienta)	4
2.2. Para JML	4
3. Alternativas a OpenJML para encontrar los mismos problemas	4
3.1. ¿Encontraría más, menos o igual cantidad de problemas?	4
3.2. ¿Encontraría los problemas antes o después que OpenJML?	5
3.3. ¿Requeriría más o menos trabajo?	5
3.4. ¿Proveería más o menos confianza sobre la seguridad del código resultante?	5

1. ¿Descubrió OpenJML todos los posibles problemas? ¿Es el código correcto después de los cambios?

1.1. Archivo Bag.java

OpenJML descubrió con precisión todos los errores de seguridad de memoria y de lógica de tamaño que el compilador de Java no detecta estáticamente. Los bugs identificados y corregidos fueron:

#	Ubicación	Error original	Descripción	Solución
1	<code>removeOnce</code> , <code>removeAll</code> , <code>getCount</code>	Condición de bucle <code>i <= n</code>	El ciclo permitía que <code>i</code> alcanzara el valor <code>n</code> , provocando un acceso a <code>contents[n]</code> , posición fuera de los límites válidos del arreglo.	Cambiar la condición de <code>i <= n</code> a <code>i < n</code> .
2	<code>add(int)</code>	Creación con <code>new int[2*n]</code>	Cuando el bag estaba vacío (<code>n == 0</code>), se creaba un arreglo de tamaño 0, impidiendo almacenar nuevos elementos.	Cambiar a <code>new int[2*n + 1]</code> para garantizar al menos una posición disponible.
3	<code>add(Bag)</code>	Desplazamiento incorrecto <code>n + 1</code> en <code>arraycopy</code>	La copia comenzaba una posición después de donde debía, dejando un hueco en la posición <code>n</code> y provocando escrituras fuera de los límites.	Reemplazar <code>n + 1</code> por <code>n</code> en la llamada a <code>arraycopy</code> .
4	<code>add(Bag)</code>	No se actualizaba <code>n</code> tras copiar	El tamaño lógico no se incrementaba, dejando los elementos añadidos invisibles para operaciones posteriores.	Actualizar con <code>n = n + b.n</code> tras la copia.
5	<code>Bag(Bag)</code>	Campos sin inicializar antes de <code>this.add(b)</code>	El constructor invocaba <code>add(b)</code> cuando <code>contents</code> aún era <code>null</code> .	Inicializar con <code>n = 0</code> y <code>contents = new int[0]</code> antes de la llamada.

Sin embargo, hay al menos un problema lógico que OpenJML **no detectó**:

`removeAll` tiene un fallo de lógica funcional. Cuando el intercambio `contents[i] = contents[n]` coloca un elemento igual a `elt` en la posición `i` , el índice `i` avanza de todas formas en la siguiente iteración, saltándose ese duplicado. Es posible que tras `removeAll(3)` sobre `{3, 5, 3}` quede un `3` sin eliminar, resultando en `{3, 5}` .

OpenJML no lo detectó porque la postcondición especificada (`n <= \old(n)`) solo habla del tamaño, no de la ausencia del elemento. Para capturarlo habría que especificar

`ensures getCount(elt) == 0`, pero OpenJML no soporta `\num_of`, lo que impidió escribir esa postcondición funcional completa.

El código es correcto para los usos habituales dentro de los límites de capacidad declarados (`n <= Integer.MAX_VALUE / 2`), pero no se puede afirmar con plena certeza que sea funcionalmente completo, porque la especificación de `removeAll` es más débil de lo deseable.

1.2. Archivo `Amount.java`

OpenJML identificó correctamente todos los errores de lógica aritmética que el compilador de Java no detecta estáticamente. Los bugs identificados y corregidos fueron:

#	Ubicación	Error original	Descripción	Solución
1	<code>negate()</code>	Parámetros invertidos	Se pasaban <code>-euros</code> y <code>-cents</code> en orden incorrecto al constructor, produciendo objetos con signo erróneo.	Corregir el orden de los parámetros en la llamada a <code>new Amount</code> .
2	<code>add()</code>	Operador incorrecto en la segunda condición	Se restaba 1 a <code>new_euros</code> cuando debía sumarse al convertir el exceso de cents a euros positivos.	Cambiar <code>-1</code> por <code>+1</code> en la actualización de <code>new_euros</code> .
3	<code>add()</code>	Límites sin cubrir los valores exactos	Las condiciones de desbordamiento no cubrían los valores exactos ± 100 , dejando representaciones inválidas como <code>Amount(n, 100)</code> sin normalizar.	Ajustar los primeros dos <code>if</code> para usar <code><-99</code> y <code>>99</code> .
4	<code>add()</code>	Condición no estricta en el cuarto <code>if</code>	Usar <code>new_cents >= 0</code> causaba correcciones innecesarias cuando el resultado era <code>(0€, 0¢)</code> , produciendo <code>(1€, -100¢)</code> .	Cambiar a <code>new_cents > 0</code> para evitar modificar el caso neutro.

El código es correcto para los usos habituales dentro de los límites declarados en los invariantes (`cents >= -99 && cents <= 99`, `euros` dentro de $\pm 10^9$), y las especificaciones son suficientes para que OpenJML verifique la consistencia de la representación interna.

2. Sugerencias de mejora para OpenJML y JML

2.1. Para OpenJML (la herramienta)

Durante esta práctica no fue posible especificar correctamente algunas propiedades relacionadas con el conteo de elementos porque OpenJML no soporta completamente `\num_of` en su proceso de verificación. Mejorar este soporte permitiría escribir especificaciones más precisas y detectar más errores lógicos.

Asimismo, los mensajes de error podrían ser más orientados al usuario. Errores como:

```
"Implicit references to 'this' are not permitted in constructor  
assignable clauses"  
"Not yet supported feature: \num_of"
```

aparecen solo en tiempo de verificación, no como advertencias del editor. Un plugin de IDE que marque las anotaciones problemáticas en tiempo real reduciría drásticamente el ciclo de corrección.

2.2. Para JML

Sintaxis más sencilla en los constructores. La distinción entre `assignable \nothing` (correcto en constructores) y `assignable field` (incorrecto porque hay un `this` previo) es contraintuitiva y no está bien documentada. Una palabra clave más descriptiva como `initializes contents, n` sería más clara.

Formas más simples de expresar conteos. Especificar propiedades relacionadas con la cantidad de veces que aparece un elemento en un arreglo debería ser más sencillo. Actualmente, las limitaciones de `\num_of` obligan a escribir especificaciones menos completas, reduciendo la capacidad de verificar ciertos comportamientos del programa.

3. Alternativas a OpenJML para encontrar los mismos problemas

Una alternativa a OpenJML sería combinar pruebas unitarias con JUnit, herramientas de análisis estático como Semgrep y SpotBugs, junto con una revisión manual del código. Las pruebas unitarias permitirían probar distintos casos (bolsa vacía, elementos duplicados, eliminación de elementos inexistentes). Las herramientas de análisis estático pueden detectar algunos errores comunes sin necesidad de especificaciones formales, mientras que la revisión manual permite identificar problemas de lógica que escapan a las herramientas automáticas.

3.1. ¿Encontraría más, menos o igual cantidad de problemas?

Probablemente encontraría una cantidad similar de problemas, aunque dependería de qué tan completas sean las pruebas. Los errores relacionados con accesos fuera de rango o arreglos mal dimensionados podrían detectarse fácilmente con los casos de prueba adecuados. Sin embargo, algunos errores lógicos más específicos, como el comportamiento de `removeAll` con elementos duplicados, podrían pasar desapercibidos si no se prueba exactamente ese escenario.

3.2. ¿Encontraría los problemas antes o después que OpenJML?

En la mayoría de los casos, los encontraría **después**. OpenJML analiza el código sin necesidad de ejecutarlo, por lo que puede detectar ciertos errores apenas se verifica el programa. Las pruebas unitarias, en cambio, solo encuentran errores cuando se ejecutan los casos correspondientes. Si un caso no fue considerado por el programador, el error podría permanecer oculto durante mucho tiempo.

3.3. ¿Requeriría más o menos trabajo?

Escribir pruebas unitarias requiere menos trabajo al inicio que escribir especificaciones completas en JML. Gran parte del tiempo de esta práctica se dedicó a entender cómo formular invariantes, precondiciones y postcondiciones para que OpenJML pudiera verificarlas correctamente. Además, arreglar un error podía producir más errores en cascada; aplicar JML en un archivo con muchas líneas de código requiere un trabajo muy tedioso y riguroso. Sin embargo, las pruebas también requieren mantenimiento y es necesario seguir agregando casos a medida que el programa evoluciona.

3.4. ¿Proveería más o menos confianza sobre la seguridad del código resultante?

Consideramos que proporcionaría **menos confianza**. Las pruebas permiten verificar que el programa funciona correctamente para los casos que se ejecutan, pero no garantizan que funcione bien para todas las situaciones posibles. OpenJML, en cambio, intenta demostrar que ciertas propiedades se cumplen para *cualquier* ejecución que respete las precondiciones definidas. Aunque esto requiere más esfuerzo, ofrece una mayor seguridad sobre la corrección del código.